CrowdFundingDS Technical Documentation

Contents

1 Smart Contracts

1.1 CommitmentToken.sol

1.1.1 Purpose

`CommitmentToken.sol` defines the token used to represent a user's stake in a crowdfunding project. When a user invests USDC into a project through `CrowdVault`, the vault mints CommitmentTokens for that user. These tokens are later used for voting, refunds, yield tracking, and AMM trading.

The important idea is:

> Each project has its own CommitmentToken ID inside one ERC-1155 contract.

For example:

- Token ID `1` represents CommitmentTokens for project `1`.
- Token ID `2` represents CommitmentTokens for project `2`.
- Token ID `3` represents CommitmentTokens for project `3`.

This avoids deploying a new ERC-20 token contract for every project.

1.1.2 ERC-1155 Design

The contract inherits from OpenZeppelin's ERC-1155 implementation:

```
import "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";

contract CommitmentToken is ERC1155 {
    ...
}
```

ERC-1155 is a multi-token standard. Unlike ERC-20, which usually represents only one token, ERC-1155 allows one smart contract to manage many token types. This is why it fits the crowdfunding model: every project can be represented by a different token ID.

The constructor calls:

```
ERC1155("")
```

The empty string means the contract is not using the ERC-1155 metadata URI mechanism directly. In this project, project metadata is stored in `CrowdVault`, not inside `CommitmentToken`.

Normally, an ERC-1155 URI could look like:

```
ERC1155("https://api.example.com/tokens/{id}.json")
```

The `{id}` placeholder is replaced by the token ID. For example, token ID `1` could point to metadata for project `1`, and token ID `2` could point to metadata for project `2`. That metadata could describe the token name, description, image, and other display information.

This project leaves the URI empty because the token contract is intentionally focused on balances, minting, burning, and transfers. The richer project data, such as project name, description, metadata URI, and additional files URL, lives in `CrowdVault`. This keeps `CommitmentToken` small and makes `CrowdVault` the source of truth for project-level information.

1.1.3 What OpenZeppelin Provides

OpenZeppelin is a library of reusable and audited smart contract building blocks. By importing and inheriting `ERC1155`, this contract receives the standard ERC-1155 behavior without reimplementing it from scratch.

```
import "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";

contract CommitmentToken is ERC1155 {
    ...
}
```

In the background, Solidity compiles the inherited OpenZeppelin logic together with this contract. That inherited logic provides functions such as:

- `balanceOf(owner, id)`
- `safeTransferFrom(from, to, id, amount, data)`
- `setApprovalForAll(operator, approved)`
- `isApprovedForAll(owner, operator)`
- `_mint(to, id, amount, data)`
- `_burn(from, id, amount)`

Conceptually, OpenZeppelin's ERC-1155 implementation stores balances like this:

```
mapping(uint256 tokenId => mapping(address owner => uint256 balance))
```

So project-specific balances behave like:

- `balances[1][Alice]` = Alice's project 1 CommitmentTokens.
- `balances[2][Alice]` = Alice's project 2 CommitmentTokens.
- `balances[1][Bob]` = Bob's project 1 CommitmentTokens.

OpenZeppelin also emits the standard ERC-1155 transfer events and performs safe-transfer receiver checks when tokens are sent to smart contracts. This is why the AMM inherits `ERC1155Holder`: it proves that the AMM contract can safely receive ERC-1155 tokens.

1.1.4 Name, Symbol, and Decimals

The contract stores simple display information:

```
string public name;
string public symbol;
uint8 public decimals = 6;
```

`name` and `symbol` identify the token in the frontend or external tools. The `decimals` value is set to `6`, matching USDC. This is useful because the system treats the relationship as:

1 USDC invested = 1 CommitmentToken minted.

Since USDC uses 6 decimals, CommitmentTokens also use 6 decimals for consistent accounting.

ERC-1155 does not standardize `decimals` in the same way ERC-20 does. Here, `decimals` is added as a public helper value so applications can interpret balances correctly.

1.1.5 Minter Role

The contract has a single authorized minter:

```
address public minter;
```

Only this address can mint or burn CommitmentTokens. The restriction is enforced by the `onlyMinter` modifier:

```
modifier onlyMinter() {
    require(msg.sender == minter, "not␣minter");
    _;
}
```

In the full system, the minter should be `CrowdVault`. A typical deployment flow is:

1. Deploy `CommitmentToken` with the deployer as the temporary minter.
2. Deploy `CrowdVault`.
3. Call `setMinter(vaultAddress)`.
4. From that point onward, only `CrowdVault` can mint or burn CommitmentTokens.

This prevents users, founders, or the AMM from creating fake project stake.

1.1.6 Changing the Minter

The minter can be changed with:

```
function setMinter(address newMinter) external onlyMinter {
    require(newMinter != address(0), "minter␣cannot␣be␣zero␣address");
    emit MinterChanged(minter, newMinter);
    minter = newMinter;
}
```

Only the current minter can transfer minter rights. The zero address is rejected to avoid accidentally leaving the contract without a valid controller.

1.1.7 Project Supply Tracking

The contract tracks total supply per project:

```
mapping(uint256 => uint256) public totalSupplyByProject;
```

This mapping answers questions like:

- How many CommitmentTokens exist for project `1`?
- How much total voting power exists for project `2`?
- How much supply should be considered when calculating refunds?

Basic ERC-1155 does not provide total supply tracking per token ID by default, so this contract stores it manually.

This means the system tracks two related but different pieces of information:

- OpenZeppelin's ERC-1155 balance tracking answers: who owns how many tokens?
- `totalSupplyByProject` answers: how many tokens exist in total for this project?

For example, ERC-1155 can answer:

```
balanceOf(Alice, 1)
```

which means "how many project 1 tokens does Alice own?"

But the vault also needs to answer:

```
totalSupplyByProject(1)
```

which means "how many project 1 tokens exist overall?"

That total is needed for vote thresholds and refund calculations. For example, if a project has 1000 CommitmentTokens in circulation and 300 tokens vote to veto, the vault can calculate that 30% of the project stake voted to veto.

OpenZeppelin also offers an `ERC1155Supply` extension that can track total supply per token ID automatically. This implementation performs that supply tracking manually through `totalSupplyByProject`.

1.1.8 Minting

The mint function is:

```
function mint(
    uint256 projectId,
    address to,
    uint256 amount
) external onlyMinter {
    totalSupplyByProject[projectId] += amount;
    _mint(to, projectId, amount, "");
    emit Mint(projectId, to, amount);
}
```

This function:

1. Checks that the caller is the minter.
2. Increases the tracked supply for the project.
3. Calls OpenZeppelin's internal ERC-1155 `_mint`.
4. Emits a `Mint` event.

Example:

> If Alice invests 100 USDC in project 1, `CrowdVault` calls `mint(1, Alice, 100)`.
> Alice now owns 100 CommitmentTokens for project 1.

The internal `_mint` function comes from OpenZeppelin's ERC-1155 contract. It is marked internal, so external users cannot call it directly. Only this contract can call it from inside its own functions.

In this line:

```
_mint(to, projectId, amount, "");
```

OpenZeppelin updates the ERC-1155 balance for `to`. Conceptually, it does:

```
balances[projectId][to] += amount;
```

It also emits the standard ERC-1155 `TransferSingle` event, which wallets, block explorers, and indexers can use to detect that tokens were created.

So the mint function does two layers of accounting:

- `totalSupplyByProject[projectId] += amount` updates this project's total supply.
- `_mint(to, projectId, amount, "")` updates the user's ERC-1155 balance.

1.1.9 Burning

The burn function is:

```
function burn(
    uint256 projectId,
    address from,
    uint256 amount
) external onlyMinter {
    totalSupplyByProject[projectId] -= amount;
    _burn(from, projectId, amount);
    emit Burn(projectId, from, amount);
}
```

This function:

1. Checks that the caller is the minter.
2. Decreases the tracked supply for the project.
3. Calls OpenZeppelin's internal ERC-1155 `_burn`.
4. Emits a `Burn` event.

Burning is used when a user receives a refund. The user gives up their project stake, and the vault returns the corresponding USDC.

Because the contract uses Solidity `0.8.20`, arithmetic underflow automatically reverts. If the burn amount is greater than the tracked supply, the subtraction fails. OpenZeppelin's `_burn` also reverts if the account does not own enough tokens.

1.1.10 Events

The contract emits three custom events:

```
event Mint(uint256 indexed projectId, address indexed to, uint256 value);
event Burn(uint256 indexed projectId, address indexed from, uint256 value);
event MinterChanged(address indexed oldMinter, address indexed newMinter);
```

These events make it easier for the frontend, indexers, and subgraphs to track token creation, token destruction, and minter changes.

1.1.11 How CrowdVault Uses CommitmentToken

`CrowdVault` uses `CommitmentToken` as the accounting and governance token for each project.

When a user invests, the vault calls:

```
commit.mint(projectId, msg.sender, amount);
```

When a user receives a refund, the vault calls:

```
commit.burn(projectId, msg.sender, amount);
```

When a user votes, the vault checks:

```
commit.balanceOf(msg.sender, projectId);
```

When the vault calculates vote thresholds or refund shares, it checks:

```
commit.totalSupplyByProject(projectId);
```

So CommitmentTokens are used for:

- investment representation,
- voting power,
- refund rights,
- yield participation,
- AMM trading.

1.1.12 AMM Compatibility

Because CommitmentTokens are ERC-1155 tokens, users can transfer them and approve operators. To sell CommitmentTokens through the AMM, a user approves the AMM with:

```
setApprovalForAll(AMM, true)
```

Then the AMM can transfer the specific project token ID from the user during a sale.

1.1.13 Important Security Notes

The token contract intentionally stays small and trusts the minter. It does not validate whether a `projectId` exists. That validation happens in `CrowdVault`. This means the correctness of minting and burning depends on the vault being the only minter.

The most important security property is:

> Only `CrowdVault` should be the minter in production.

If the wrong address controls `minter`, that address could mint fake CommitmentTokens or burn user balances.

1.1.14 Summary

`CommitmentToken.sol` is a multi-project ERC-1155 token contract. Each project uses its `projectId` as the ERC-1155 token ID. The contract allows only one authorized minter, normally `CrowdVault`, to mint and burn tokens. These tokens represent user stake in projects and are used across the system for voting, refunds, yield accounting, and AMM trading.

1.2 RevenueRouter.sol

1.2.1 Purpose

`RevenueRouter.sol` handles platform revenue. In the current design, platform revenue comes from release fees taken by `CrowdVault` when milestone funds are released.

The simple flow is:

> `CrowdVault` takes a release fee, sends that fee to `RevenueRouter`, and the router lets the admin collect it later.

This makes `RevenueRouter` a small platform-fee vault. It does not distribute revenue to backers. It receives USDC, tracks how much revenue has arrived, and allows the platform owner/admin to collect the available amount.

1.2.2 USDC Interface

The contract defines a small ERC-20 interface:

```
interface IERC20Pay {
    function transferFrom(address from, address to, uint256 amount) external returns (
        bool);
    function transfer(address to, uint256 amount) external returns (bool);
}
```

The router only needs two token operations:

- `transferFrom`: pull revenue from `CrowdVault`.
- `transfer`: send collected revenue to the admin.

The actual token used is USDC. The token address is passed once during deployment and stored as:

```
IERC20Pay public immutable usdc;
```

Because `usdc` is immutable, the router always knows which token it receives and pays out.

1.2.3 Admin

The router stores:

```
address public admin;
```

The admin is set in the constructor:

```
constructor(address usdc_) {
    usdc = IERC20Pay(usdc_);
    admin = msg.sender;
}
```

This means the deployer of `RevenueRouter` becomes the platform owner/admin. Only this address can collect revenue.

The admin restriction is enforced by:

```
modifier onlyAdmin() {
    if (msg.sender != admin) revert NotAdmin();
    _;
}
```

1.2.4 Revenue Tracking

The contract tracks two totals:

```
uint256 public totalRevenueReceived;
uint256 public totalCollected;
```

`totalRevenueReceived` means the total amount of USDC revenue that has entered the router over its lifetime.

`totalCollected` means the total amount of USDC that the admin has already collected.

The currently collectable amount is:

```
totalRevenueReceived - totalCollected
```

1.2.5 onRevenue

Revenue enters through:

```
function onRevenue(uint256 amount) external {
    if (amount == 0) revert ZeroAmount();
    if (!usdc.transferFrom(msg.sender, address(this), amount)) revert TransferFailed()
        ;
    totalRevenueReceived += amount;
    emit RevenueReceived(amount);
}
```

This function is called by `CrowdVault` after the vault calculates a release fee.

The function receives only an `amount`, not a token address, because the router already knows the revenue token:

```
IERC20Pay public immutable usdc;
```

So calling:

```
onRevenue(10_000_000)
```

means:

> Pull `10_000_000` units of the stored USDC token from the caller into this router.

Since USDC uses 6 decimals, `10_000_000` represents 10 USDC.

The actual token transfer happens here:

```
usdc.transferFrom(msg.sender, address(this), amount)
```

In normal use, `msg.sender` is `CrowdVault`. Before calling `onRevenue`, the vault approves the router to pull the fee:

```
usdc.approve(revenueRouter, fee);
IRevenueRouter(revenueRouter).onRevenue(fee);
```

The full revenue movement is:

> `CrowdVault` approves the fee, `RevenueRouter` pulls the USDC, and `totalRevenueReceived` increases.

1.2.6 Collecting Revenue

The admin collects revenue with:

```
function collect() external onlyAdmin {
    uint256 toPay = totalRevenueReceived - totalCollected;
    if (toPay == 0) revert NothingToClaim();
    totalCollected += toPay;
    if (!usdc.transfer(admin, toPay)) revert TransferFailed();
    emit Collected(admin, toPay);
}
```

This function:

1. Calculates uncollected revenue.
2. Reverts if there is nothing to collect.
3. Updates `totalCollected`.

4. Transfers USDC to the admin.
5. Emits a `Collected` event.

Example:

If the router received 100 USDC and the admin already collected 40 USDC, then `collect()` sends 60 USDC to the admin.

1.2.7 How CrowdVault Uses RevenueRouter

`CrowdVault` calls `RevenueRouter` only when release fees are enabled.

During a milestone release, `CrowdVault` calculates:

```
uint256 fee = (amt * releaseFeeBps) / 10_000;
uint256 toTreasury = amt - fee;
```

The project treasury receives `toTreasury`. The fee is routed to `RevenueRouter`:

```
_withdrawAndTransfer(address(this), fee);
usdc.approve(revenueRouter, fee);
IRevenueRouter(revenueRouter).onRevenue(fee);
```

So the project still receives most of the release, while the platform receives the configured fee.

1.2.8 Why the Router Stores the Token Address

The router does not accept arbitrary revenue tokens. It is designed for one token: USDC.

That is why `onRevenue` only takes:

```
uint256 amount
```

instead of:

```
address token, uint256 amount
```

This is simpler and safer. The router cannot accidentally receive an unsupported token through `onRevenue`; it always pulls the configured USDC token.

1.2.9 Summary

`RevenueRouter.sol` is a platform revenue collector. It receives USDC release fees from `CrowdVault`, tracks total received revenue, tracks total collected revenue, and allows only the admin to collect the available USDC.

1.3 MockLender.sol

1.3.1 Purpose

`MockLender.sol` is a simplified lending module used by `CrowdVault`. It receives USDC from the vault, tracks deposited principal, allows a fixed small amount of mock yield to be added, and lets only the vault withdraw principal or yield.

Even though the contract is called `MockLender`, the token it holds can still be real Sepolia USDC. The word "mock" means the lending behavior is simplified. It is not a full external lending protocol like Aave or Compound.

The high-level flow is:

Backers invest USDC into `CrowdVault`. The vault supplies that USDC to `MockLender`. A fixed amount of mock yield can be added. The vault harvests yield and later lets backers claim it through `CrowdVault.claimYield()`.

1.3.2 USDC Interface

The contract defines a small ERC-20 interface:

```
interface IERC20Like {
    function transferFrom(address from, address to, uint256 amount) external returns (
        bool);
    function transfer(address to, uint256 amount) external returns (bool);
    function balanceOf(address who) external view returns (uint256);
    function approve(address spender, uint256 amount) external returns (bool);
}
```

`MockLender` uses this interface to interact with USDC. It needs:

- `transferFrom`: pull USDC into the lender.
- `transfer`: send USDC back to the vault.
- `balanceOf`: check real token liquidity before withdrawals.
- `approve`: included in the interface, although this contract does not currently call it internally.

1.3.3 State Variables

The simplified lender has these main variables:

```
IERC20Like public immutable usdc;
uint256 public constant YIELD_AMOUNT = 10_000; // 0.01 USDC
address public admin;
address public withdrawer;
uint256 public totalSupplied;
uint256 public accruedYield;
```

`usdc` stores the ERC-20 token used by the lender. It is immutable, so it is set once during deployment and cannot be changed later.

`YIELD_AMOUNT` is the fixed amount of mock yield added each time `addYield()` is called. Because USDC uses 6 decimals, `10_000` represents:

0.01 USDC

`admin` is the address that deployed the lender. The admin can set the withdrawer and add mock yield.

`withdrawer` is the only address allowed to withdraw funds from the lender. In the system, this should be `CrowdVault`.

`totalSupplied` tracks deposited principal. Principal means original deposited money, not profit.

`accruedYield` tracks yield that has been added and can be harvested by the vault.

1.3.4 Roles

There are two roles:

- `admin`: configures the lender and can add mock yield.
- `withdrawer`: can withdraw principal and yield. This should be `CrowdVault`.

The admin restriction is:

```
modifier onlyAdmin() {
    require(msg.sender == admin, "not␣admin");
    _;
}
```

The withdrawer restriction is:

```
modifier onlyWithdrawer() {
    require(msg.sender == withdrawer, "not␣withdrawer");
    _;
}
```

This design prevents normal users, founders, or random callers from withdrawing directly from the lender. All withdrawals should go through `CrowdVault`, where the project rules are enforced.

1.3.5 Constructor

The constructor is:

```
constructor(address usdc_) {
    usdc = IERC20Like(usdc_);
    admin = msg.sender;
}
```

It stores the USDC token address and sets the deployer as admin.

1.3.6 setWithdrawer

`setWithdrawer` chooses the address allowed to withdraw:

```
function setWithdrawer(address withdrawer_) external onlyAdmin {
    require(withdrawer_ != address(0), "withdrawer=0");
    withdrawer = withdrawer_;
    emit WithdrawerSet(withdrawer_);
}
```

After deployment, the expected setup is:

```
lender.setWithdrawer(vaultAddress)
```

Without this step, `CrowdVault` cannot call `withdraw()` or `withdrawYield()`.

1.3.7 supply

`supply` receives principal deposits:

```
function supply(uint256 amount) external {
    require(amount > 0, "amount=0");
    require(usdc.transferFrom(msg.sender, address(this), amount), "tf");
    totalSupplied += amount;
    emit Supplied(msg.sender, amount);
}
```

In normal use, `msg.sender` is `CrowdVault`. The vault approves the lender, then the lender pulls USDC from the vault.

The function increases:

```
totalSupplied
```

because supplied funds are principal.

1.3.8 withdraw

`withdraw` lets the withdrawer pull funds out:

```
function withdraw(uint256 amount, address to) external onlyWithdrawer {
    require(to != address(0), "to=0");
    require(amount <= totalSupplied + accruedYield, "bal");
    if (amount <= totalSupplied) {
        totalSupplied -= amount;
    } else {
        uint256 fromYield = amount - totalSupplied;
        totalSupplied = 0;
        accruedYield = accruedYield > fromYield ? accruedYield - fromYield : 0;
    }
    require(usdc.balanceOf(address(this)) >= amount, "liq");
    require(usdc.transfer(to, amount), "t");
    emit Withdrawn(to, amount);
}
```

This is used by `CrowdVault` when it needs USDC back from the lender.
Examples include:

- releasing milestone funds to a project treasury,
- refunding backers,
- seeding the AMM,
- paying claimable yield if the vault needs liquidity.

The function first reduces `totalSupplied`. If the requested amount is larger than the remaining principal, it reduces `accruedYield` as well.

It also checks:

```
usdc.balanceOf(address(this)) >= amount
```

This verifies that the lender actually holds enough USDC tokens, not just accounting values.

1.3.9 withdrawYield

`withdrawYield` lets the vault withdraw only yield:

```
function withdrawYield(uint256 amount, address to) external onlyWithdrawer {
    require(to != address(0), "to=0");
    require(amount <= accruedYield, "yield");
    accruedYield -= amount;
    require(usdc.balanceOf(address(this)) >= amount, "liq");
    require(usdc.transfer(to, amount), "t");
    emit YieldWithdrawn(to, amount);
}
```

`CrowdVault.harvestYield()` uses this function. It is stricter than `withdraw()` because it cannot touch principal. It only withdraws from `accruedYield`.

1.3.10 balance

The accounting balance is:

```
function balance() external view returns (uint256) {
    return totalSupplied + accruedYield;
}
```

`CrowdVault` uses this to compare lender accounting against the principal that should still be locked. If the lender balance is greater than locked principal, the difference can be treated as yield.

1.3.11 addYield

`addYield` adds a fixed amount of mock yield:

```
function addYield() external onlyAdmin {
    require(usdc.transferFrom(msg.sender, address(this), YIELD_AMOUNT), "tf");
    accruedYield += YIELD_AMOUNT;
    emit YieldAdded(YIELD_AMOUNT);
}
```

This function no longer takes an arbitrary amount. It always uses:

```
YIELD_AMOUNT
```

which is `0.01` USDC.

The admin must first approve the lender to pull `YIELD_AMOUNT` USDC. Then `addYield()` pulls that USDC and increases `accruedYield`.

The flow is:

> Admin approves `0.01` USDC to `MockLender`. Admin calls `addYield()`. `MockLender` pulls the USDC and records it as yield.

1.3.12 How Backers Receive Yield

Backers do not call `MockLender` directly.

The complete flow is:

1. Backers invest USDC in `CrowdVault`.
2. `CrowdVault` supplies the USDC to `MockLender`.
3. Admin adds mock yield with `MockLender.addYield()`.
4. Someone calls `CrowdVault.harvestYield()`.
5. `CrowdVault` calls `MockLender.withdrawYield(...)`.
6. Backers call `CrowdVault.claimYield()`.

So the user-facing yield functions are on `CrowdVault`, not on `MockLender`.

1.3.13 Summary

`MockLender.sol` is a simplified USDC lending module. It stores vault-supplied principal in `totalSupplied`, stores manually added yield in `accruedYield`, allows only `CrowdVault` to withdraw, and uses a fixed `YIELD_AMOUNT` of 0.01 USDC for mock yield generation.

1.4 CommitmentAMM.sol

1.4.1 Purpose

`CommitmentAMM.sol` is the trading contract for CommitmentTokens. It lets users buy or sell a project's CommitmentToken using USDC.

AMM stands for Automated Market Maker. An AMM is a smart contract that lets users trade against a liquidity pool instead of trading directly with another person.

In a normal order book market, one user needs to place a buy order and another user needs to place a sell order. In an AMM, the pool itself is always the counterparty.

In this project:

- Users buy CommitmentTokens by sending USDC to the AMM.
- Users sell CommitmentTokens by sending CommitmentTokens to the AMM.
- The AMM stores pool balances for each project.
- The AMM price changes automatically based on pool balances.

1.4.2 How This AMM Is Tailored for the Project

This AMM is not a general-purpose exchange like Uniswap. It is tailored to the crowdfunding system.

Each crowdfunding project has its own CommitmentToken ID. Because of that, the AMM stores separate pools per project:

```
mapping(uint256 => uint256) public poolUsdc;
mapping(uint256 => uint256) public poolCommit;
mapping(uint256 => bool) public seeded;
```

For example:

- `poolUsdc[1]` stores the USDC liquidity for project `1`.
- `poolCommit[1]` stores the CommitmentToken liquidity for project `1`.
- `seeded[1]` tells us whether project `1`'s AMM pool has been created.

So there is one AMM contract, but many project-specific pools inside it.

1.4.3 Important Contract References

The AMM connects to three other contracts:

```
IERC20Swap public immutable usdc;
ICommitSwap public immutable commit;
ICrowdVault public immutable vault;
```

`usdc` is the USDC token used for buying and selling.

`commit` is the CommitmentToken contract. The AMM calls it when it needs to transfer ERC-1155 CommitmentTokens.

`vault` is the CrowdVault contract. The vault is the only contract allowed to seed AMM pools.

1.4.4 Admin Variables

The AMM still has an admin:

```
address public admin;
address public pendingAdmin;
```

The admin can manage settings such as the AMM fee and admin transfer. The admin can no longer manually seed pools and can no longer remove liquidity.

This is intentional. In the current design, AMM liquidity should come from the project flow, not from manual admin actions.

1.4.5 Trading Fee

The AMM fee is stored in basis points:

```
uint256 public feeBps = 30;
```

`bps` means basis points. One basis point is `0.01%`.

So:

- `30 bps` = `0.30%`
- `100 bps` = `1.00%`
- `1000 bps` = `10.00%`

The fee can be changed by the admin:

```
function setFee(uint256 feeBps_) external onlyAdmin {
    if (feeBps_ > 1000) revert FeeTooHigh();
    feeBps = feeBps_;
    emit FeeSet(feeBps_);
}
```

The maximum fee is `1000 bps`, meaning `10%`.

1.4.6 Modifiers

The AMM uses modifiers to keep checks readable.

```
modifier onlyVault() {
    if (msg.sender != address(vault)) revert NotVault();
    _;
}
```

`onlyVault` means the function can only be called by `CrowdVault`.

```
modifier validProject(uint256 projectId) {
    if (projectId == 0 || projectId > vault.projectCount())
        revert BadProject();
    _;
}
```

`validProject` checks that the project ID exists.

```
modifier projectTradable(uint256 projectId) {
    if (!vault.ammProjectReady(projectId)) revert ProjectNotTradable();
    _;
}
```

`projectTradable` checks that the project is allowed to trade.

```
modifier notSeeded(uint256 projectId) {
    if (seeded[projectId]) revert AlreadySeeded();
    _;
}
```

`notSeeded` prevents the same project pool from being seeded twice.

```
modifier seededProject(uint256 projectId) {
    if (!seeded[projectId]) revert NotSeeded();
    _;
}
```

`seededProject` makes sure users cannot swap before the pool exists.

```
modifier nonEmptySeed(uint256 usdcIn, uint256 commitIn) {
    if (usdcIn == 0 || commitIn == 0) revert EmptyPool();
    _;
}
```

`nonEmptySeed` makes sure the initial AMM pool does not start with zero USDC or zero CommitmentTokens.

```
modifier nonZeroAmount(uint256 amount) {
    if (amount == 0) revert ZeroAmount();
    _;
}
```

`nonZeroAmount` makes sure users cannot make a zero-sized swap.

1.4.7 Automatic Seeding From CrowdVault

The AMM pool is created through:

```
function seedFromVault(
    uint256 projectId,
    uint256 usdcIn,
    uint256 commitIn
)
    external
    onlyVault
    validProject(projectId)
    projectTradable(projectId)
    notSeeded(projectId)
    nonEmptySeed(usdcIn, commitIn)
{
    poolUsdc[projectId] = usdcIn;
    poolCommit[projectId] = commitIn;
    seeded[projectId] = true;

    emit Seeded(projectId, usdcIn, commitIn);
}
```

This function is called by `CrowdVault`, not by users and not by the admin.
The intended flow is:

1. Backers fund a project.
2. The project reaches the right stage.

3. The founder claims the first milestone.

4. `CrowdVault` releases milestone funds.

5. `CrowdVault` seeds the AMM by calling `seedFromVault(...)`.

The AMM no longer has manual admin seeding. The old manual `seed(...)` function was removed because it duplicated the real product flow and could confuse the system.

1.4.8 Why removeLiquidity Was Removed

The old `removeLiquidity(...)` function allowed the admin to pull liquidity out of the AMM.

That was useful for emergency maintenance, but it also gave the admin too much control. If the admin could remove liquidity, users might lose the ability to sell into the AMM.

The current design removes that function. Once `CrowdVault` seeds the pool, the liquidity stays in the AMM and users trade against it.

1.4.9 Buying CommitmentTokens

Users buy CommitmentTokens with:

```
function swapUsdcForCommit(
    uint256 projectId,
    uint256 usdcIn,
    uint256 minCommitOut
)
    external
    nonZeroAmount(usdcIn)
    seededProject(projectId)
    projectTradable(projectId)
    returns (uint256 out)
```

The flow is:

User sends USDC to AMM. AMM sends CommitmentTokens to user.

The important transfer is:

```
if (!usdc.transferFrom(msg.sender, address(this), usdcIn))
    revert TransferFailed();
```

This line checks the user's real USDC balance indirectly. If the user claims they want to swap `1000` USDC but only has `5` USDC, `transferFrom` fails and the whole transaction reverts.

It also checks allowance indirectly. The user must approve the AMM before the AMM can pull their USDC.

So the vault does not check the user's balance. The USDC token contract enforces the real balance and allowance.

1.4.10 Selling CommitmentTokens

Users sell CommitmentTokens with:

```
function swapCommitForUsdc(
    uint256 projectId,
    uint256 commitIn,
    uint256 minUsdcOut
)
    external
```

```
    nonZeroAmount(commitIn)
    seededProject(projectId)
    projectTradable(projectId)
    returns (uint256 out)
```

The flow is:

User sends CommitmentTokens to AMM. AMM sends USDC to user.

The important ERC-1155 transfer is:

```
commit.safeTransferFrom(
    msg.sender,
    address(this),
    projectId,
    commitIn,
    ""
);
```

This calls the CommitmentToken contract. Even though the call goes through `commit`, the tokens move from:

`msg.sender` to `address(this)`

That means:

user wallet to AMM contract

The AMM does not manually check `balanceOf(user, projectId)` first. The ERC-1155 `safeTransferFrom` function already checks whether the user has enough tokens.

It also checks whether the user approved the AMM with:

```
setApprovalForAll(ammAddress, true)
```

If the user does not have enough CommitmentTokens, or did not approve the AMM, `safeTransferFrom` reverts and the swap fails.

1.4.11 Fake Amounts Cannot Trick the AMM

A user can type any amount into the frontend, but the contract only succeeds if the tokens can actually move.

For USDC buys:

```
usdc.transferFrom(msg.sender, address(this), usdcIn)
```

fails if the user does not have enough USDC or did not approve enough USDC.

For CommitmentToken sells:

```
commit.safeTransferFrom(msg.sender, address(this), projectId, commitIn, "")
```

fails if the user does not have enough CommitmentTokens or did not approve the AMM.

So a real wallet address with a fake amount does not allow stealing. The transaction simply reverts.

1.4.12 The getAmountOut Formula

The AMM calculates swap output with:

```
function getAmountOut(
    uint256 amountIn,
    uint256 reserveIn,
    uint256 reserveOut
) public view returns (uint256) {
    if (reserveIn == 0 || reserveOut == 0) revert EmptyPool();
    uint256 inWithFee = (amountIn * (10_000 - feeBps)) / 10_000;
    return (inWithFee * reserveOut) / (reserveIn + inWithFee);
}
```

This uses the constant-product AMM idea:

```
x * y = k
```

Where:

- `x` is one side of the pool,
- `y` is the other side of the pool,
- `k` is the constant the AMM tries to preserve.

For example, if the pool has:

- `1000` USDC,
- `1000` CommitmentTokens,

then:

```
x * y = 1000 * 1000 = 1,000,000
```

If a user buys CommitmentTokens with `100` USDC, the AMM does not simply give exactly `100` CommitmentTokens. The user's own buy changes the pool balance and therefore changes the price.

First, the AMM applies the fee:

```
inWithFee = amountIn * (10_000 - feeBps) / 10_000
```

With a `30 bps` fee:

```
inWithFee = 100 * 9970 / 10000 = 99.7
```

Then the output is:

```
amountOut = inWithFee * reserveOut / (reserveIn + inWithFee)
```

Using the example:

```
amountOut = 99.7 * 1000 / (1000 + 99.7)
amountOut = 90.66 CommitmentTokens approximately
```

The user receives less than `100` CommitmentTokens because their trade moves the price. This is called slippage.

1.4.13 Slippage Protection

Both swap functions include a minimum output parameter:

```
uint256 minCommitOut
uint256 minUsdcOut
```

After the AMM calculates output, it checks:

```
if (out < minCommitOut) revert Slippage();
if (out < minUsdcOut) revert Slippage();
```

This protects users from receiving much less than expected. The frontend can calculate an expected output and set a minimum acceptable amount.

1.4.14 Does the AMM Guarantee Profit?

No. The AMM does not guarantee profit for early buyers.

If a user buys early and many people buy after them, the AMM price may go up. If the first user later sells, they may receive more USDC than they originally paid.

But this is not guaranteed.

Profit depends on:

- how much other users buy,
- how much other users sell,
- AMM fees,
- pool liquidity,
- slippage,
- the timing of the user's exit.

The AMM only guarantees the formula. It does not guarantee that a trader will make money.

1.4.15 Summary

`CommitmentAMM.sol` is the secondary trading contract for project CommitmentTokens. It stores one USDC/CommitmentToken pool per project, allows only `CrowdVault` to seed pools automatically, lets users buy and sell against those pools, and calculates prices using a constant-product AMM formula. Manual admin seeding and admin liquidity removal were removed so the AMM follows the real project flow more clearly.